

Solidity智能合约开发

从入门到精通

第3.2课

映射 和 结构体

Mapping and Struct

今天的学习内容



1. Mapping（映射）基础

键值对存储结构， $O(1)$ 查找速度



2. Mapping的特性与限制

五大特性、默认值机制、主要限制



3. 嵌套Mapping

多维度键值对存储结构



4. Struct（结构体）定义

自定义复合数据类型



5. Struct的使用

创建方式、存储位置、访问修改



6. Mapping + Struct组合模式

最强大的数据结构模式之一



7. 实战应用案例

用户管理系统、投票系统等

Mapping (映射)

📖 概念

键值对 (key-value pair) 存储结构，类似于其他编程语言中的哈希表 (HashMap) 或字典 (Dictionary)

根据键快速查找对应的值，实现 $O(1)$ 的常数时间查找

💻 语法

```
mapping(keyType => valueType) 变量名;
```

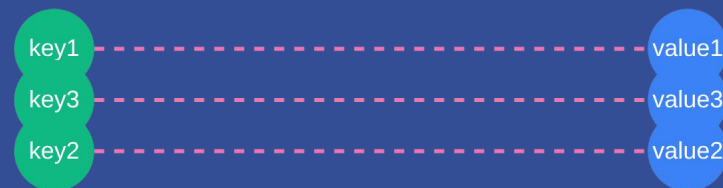
- **keyType**: 键的类型 (基本类型, 如address、uint等)
- **valueType**: 值的类型 (任何类型, 包括复杂类型)

💡 常见示例

```
mapping(address => uint) balances; 存储每个地址的余额
```

```
mapping(uint => string) names; 将ID映射到名称
```

🔄 工作原理

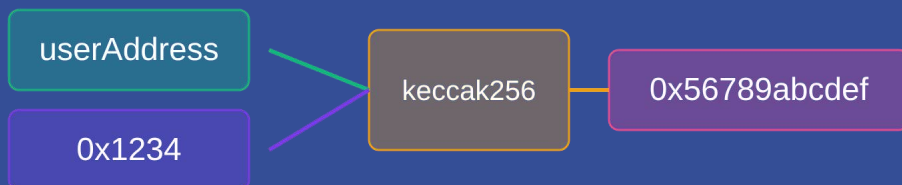


Mapping的底层原理

存储机制

Mapping不直接存储键值对，而是通过哈希函数计算存储位置。

存储位置 = keccak256(key, mappingSlot)



查找过程

- 1 输入键 (**key**)
提供要查找的键
- 2 计算哈希值
使用keccak256哈希函数
- 3 定位存储位置
根据哈希值定位存储位置
- 4 读取值
从存储位置读取对应值

🕒 时间复杂度: **O(1)** 常数时间查找

Mapping的重要特性



所有键都"存在"

未设置的键返回其valueType的默认值（如uint为0，bool为false）

不会因为键不存在而报错



不存储键

Mapping只存储值，不存储键本身
因此，无法直接获取所有键的列表



不能遍历

无法使用循环（如for循环）遍历所有键值对
没有.length属性来获取Mapping的大小



只能用于storage

Mapping必须是状态变量，存储在合约的storage中

不能在函数内部创建Mapping，也不能在memory或calldata中使用



不能作为参数/返回值

Mapping不能直接作为函数的参数传递
也不能作为函数的返回值

这些特性决定了Mapping的使用方式和限制，是Solidity数据结构设计的基础

默认值机制

💡 重要概念

在Solidity中，Mapping的所有可能键都"存在"并预先映射到其valueType的默认值。

```
</> mapping(address => uint) balances;
```

查询任何未曾设置过的地址，例如**balances[0x123...]**，都将返回**0**。

⚠️ 问题

如何区分一个值为0的字段是真实设置的，还是从未设置过的默认值？

📍 不同类型的默认值

uint

→ 0

bool

→ false

address

→ 0x0000...

string

→ ""

✅ 解决方案

通常通过在结构体中添加一个**bool exists**字段来明确标记记录是否存在。

</> 示例:

```
struct User {  
    string name;  
    uint balance;  
    bool exists; // 明确标记是否存在  
}
```

Mapping的限制

⚠ 主要限制



不能遍历

无法使用for循环遍历所有键值对



不能获取长度

没有.length属性来获取Mapping的大小



不能删除整个Mapping

只能通过delete map[key]删除单个键的值



不能作为函数参数

function test(mapping(...) map) // 错误



不能作为返回值

function get() returns (mapping(...)) // 错误



不能在memory中使用

Mapping只能是storage变量



解决方案

为了克服不能遍历和获取长度的限制，通常会配合数组（Array）来存储键的列表，实现可遍历的Mapping。

嵌套Mapping (Nested Mapping)

</> 语法与理解

```
mapping(keyType1 => mapping(keyType2 => valueType))
```

嵌套Mapping允许在Mapping的值类型中再包含一个Mapping
构建多维度的键值对存储结构

🔗 结构可视化

外层mapping
keyType1 => ...



内层mapping
keyType2 => valueType

💰 示例1: 多代币余额

```
mapping(address => mapping(address => uint))  
tokenBalances;
```

- 用户地址(keyType1)对应着一个代币地址(keyType2)到余额数量(valueType)的映射
- 访问方式: tokenBalances[user][token] = 1000;
- 查询方式: uint amount = allowance[owner][spender];

🛡️ 示例2: ERC20授权

```
mapping(address => mapping(address => uint)) allowance;
```

- 记录所有者(keyType1)授权给被授权者(keyType2)可以花费的代币数量(valueType)
- 用于ERC20代币的授权机制
- 是DeFi协议中重要的安全控制组件

ERC20代币授权机制

授权模式



数据结构

```
mapping(address => mapping(address => uint)) allowance;
```

所有者 → 被授权者 → 授权数量

操作流程

1 Alice调用approve(Bob, 100)

```
allowance[Alice][Bob] = 100
```

Bob调用transferFrom(Alice, Carol, 50)

检查: $\text{allowance}[\text{Alice}][\text{Bob}] \geq 50$

2 扣除: $\text{allowance}[\text{Alice}][\text{Bob}] -= 50$

转账: 从Alice转50给Carol

应用场景

↔ 交易所充值

📈 DeFi协议

💰 代币支付

🪙 空投分发

Mapping + Array 组合模式

问题: Mapping不能遍历, 无法获取长度 → 解决方案: 配合Array使用



Mapping

balances

mapping(address => uint)

balances[user] → O(1)查找



Array

userList

address[]

userList → 遍历所有用户



Exists Check

isUser

mapping(address => bool)

isUser[user] → O(1)检查存在



快速查找



可遍历



存在检查



协同工作

// 添加用户

```
balances[user] = amount;    userList.push(user);    isUser[user] = true;
```

// 遍历所有用户

```
for(user in userList) { // 遍历 }
```

// 检查用户是否存在

Mapping + Array 架构

Mapping

```
mapping(address => uint) balances;
```

存储数据值
O(1)快速查找

Array

```
address[] public userList;
```

存储所有键
支持遍历操作

组合优势

既支持O(1)快速查找
又支持遍历所有数据

添加用户流程

```
balances[user] =  
amount
```

```
userList.push(user)
```

```
isUser[user] = true
```

添加完成

查询流程

```
balances[user]  
O(1)快速查找
```

返回余额

```
for(userList)  
遍历所有用户
```

处理用户列表

这个组合模式是Solidity数据结构设计中的核心模式，既保留了Mapping的快速查找优势，又通过Array弥补了其不可遍历的不足

Struct (结构体)



概念

结构体是Solidity中自定义的复合数据类型，它允许开发者将不同类型的相关数据组织在一起，形成一个逻辑实体。



作用

- 将相关数据组织在一起
- 提高代码的可读性
- 增强数据管理效率
- 形成逻辑完整的实体



定义语法

```
struct 结构体名 {  
    类型1 字段1;  
    类型2 字段2;  
    // ...  
}
```



示例

```
struct User {  
    string name;  
    uint age;  
    address wallet;  
    bool isActive;  
}
```

User 结构体

Struct定义位置



合约内部

结构体直接在合约内部定义，作用域仅限于当前合约及其继承合约。

```
contract MyContract {
    struct User {
        string name;
        uint age;
    }
}
```



合约外部（全局）

结构体在合约外部、文件的顶层定义，可在同一文件中的所有合约中被引用。

```
struct User {
    string name;
    uint age;
}
contract MyContract {
    // 可以使用User
}
```



库文件中

对于复杂项目，推荐将结构体定义在独立的库文件或接口中，提高复用性。

```
library Types {
    struct User {
        string name;
        uint age;
    }
}
```



最佳实践建议

- ✓ 对于复杂项目，将Struct定义在库文件或接口中
- ✓ 提高Struct的复用性，使项目结构更加清晰

创建Struct实例

方式1: 逐个赋值

```
User storage user = users[0];
user.name = "Alice";
user.age = 25;
user.wallet = msg.sender;
```

- ✓ 适用于灵活部分更新
- ✓ 逐步填充数据
- ✗ 代码冗长

方式2: 构造器语法

```
User memory user = User("Bob", 30, msg.sender, true);
```

- ✓ 简洁明了
- ⚠ 需记住字段顺序
- ✗ 易错性高

方式3: 键值对语法

推荐

```
User memory user = User({
    name: "Charlie",
    age: 35,
    wallet: msg.sender,
    isActive: true
});
```

- ✓ 清晰明确
- ✓ 不依赖字段顺序
- ✓ 可读性最好

💡 在实际开发中，键值对语法是最推荐的创建方式

Struct的存储位置



Storage

- ✓ 持久化存储在区块链上
- ✓ 状态变量、映射的值、动态数组元素
- ⚠ 读写Gas成本较高

```
User public admin;           // 状态变量
User[] public users;         // 数组元素
mapping(address => User) usersMap; // 映射值
```



Memory

- ✓ 函数执行期间的临时存储
- ✓ 函数执行结束后数据销毁
- ⚠ Gas成本较低

```
function test(User memory user) public {
    User memory temp = User("Guest", 18,
address(0), false);
}
```



Calldata

- ✓ 外部函数参数的只读区域
- ✓ 数据不可修改
- ⚠ Gas成本最低

```
function test(User calldata user) external {
    // user是只读的，不能修改
}
```



特殊限制

如果结构体中包含**mapping**类型，则该结构体实例**只能**在**storage**中创建和使用，不能在**memory**或**calldata**中。

访问和修改Struct

访问字段

```
// 假设有一个User结构体
string userName = user.name;
uint userAge = user.age;
address userWallet = user.wallet;
```

修改字段

```
users[0].age = 30;
users[0].isActive = false;
// 只修改了内存中的副本, users[0]在链上的age不变
```

整体替换

```
users[0] = User({
  name: "NewName",
  age: 25,
  wallet: addr,
  isActive: true
});
```

Storage引用与Memory复制

Storage引用

```
User storage userRef = users[0];
userRef.age = 30; // 直接修改链上数据
```

Memory复制

```
User memory userCopy = users[0];
userCopy.age = 30; // 只修改内存中的副本
```

💡在Solidity中, struct的存储位置决定了数据的持久性和修改行为

Mapping + Struct 组合模式



组合数据结构

最强大的数据结构模式之一

基本模式

```
struct UserInfo {  
    string name;  
    uint balance;  
    bool exists; // 重要  
}  
  
mapping(address => UserInfo) users;
```

优势

- ✓ Mapping提供O(1)的查找速度
- ✓ Struct能够组织多个相关字段
- ✓ **exists**字段解决默认值混淆问题

应用场景

- 👤 用户管理系统
- 💰 资产信息管理
- 🔗 复杂实体关系建模

exists字段的重要性

⚠ 问题：默认值混淆

在Solidity的Mapping中，查询任何一个键都会返回其值类型的默认值：

- `uint` → 0
- `bool` → `false`
- `address` → 0x0000...
- `string` → ""

// 问题示例

```
mapping(address => uint) balances;
```

// 无法区分：

```
balances[user] // 返回0，可能是因为：
```

```
// 1. 用户余额确实是0
```

```
// 2. 用户从未在合约中注册
```

✅ 解决方案：exists字段

最佳实践是在Struct中添加一个`bool exists`字段：

// 推荐的结构

```
struct UserInfo {  
    uint balance;  
    bool exists;  
}
```



用户存在
`exists = true`



安全访问
`if (users[addr].exists)`

// 实际应用示例

```
struct UserInfo {  
    string name;  
    uint balance;  
    bool exists;  
}
```

完整的Mapping+Struct+Array组合

数据结构设计

Struct定义

```
struct UserInfo {  
    string name;  
    uint balance;  
    uint registeredAt;  
    bool exists;  
}
```

主存储Mapping

```
mapping(address => UserInfo) public users;
```

键列表Array

```
address[] public userAddresses;
```

计数器

```
uint public userCount;
```

功能实现

注册用户

```
users[addr] = UserInfo({  
    name: name,  
    balance: 0,  
    registeredAt: block.timestamp,  
    exists: true  
});  
  
userAddresses.push(addr);  
userCount++;
```

查询用户

```
users[addr]; // O(1)快速查找
```

遍历用户

```
for (address user : userAddresses) {  
    // 处理每个用户  
}
```

检查存在

```
users[addr].exists; // 布尔值
```

Struct中包含Mapping

✓ 允许的方式

```
// Struct可以包含Mapping字段
struct Proposal {
    string description;
    uint voteCount;
    mapping(address => bool) voters;
}
```

⊘ 限制条件

- 只能在storage中声明和使用
- 不能作为函数参数或返回值
- 不能在memory或calldata中创建

🔑 访问方式

```
Proposal storage p = proposals[id];
p.voters[msg.sender] = true;
// 检查地址是否已投票
bool hasVoted = p.voters[msg.sender];
```

💡 应用场景

🗳️ 投票系统

一个提案需要记录哪些地址已投票

👤 权限管理

记录哪些地址有特定权限

📖 Struct中包含Mapping是一种强大的模式，用于管理实体内部的复杂关系

常见的数据结构模式



模式1：用户管理

```
mapping(address => UserInfo) users;  
address[] userList;
```

通过地址快速查找用户信息，同时维护用户地址列表以支持遍历



模式2：ID索引

```
mapping(uint => Item) items;  
uint public itemCount;
```

通过自增ID索引项目，记录项目总数以生成新的唯一ID



模式3：双向映射

```
mapping(address => uint) addressToId;  
mapping(uint => address) idToAddress;
```

将地址映射到唯一ID，再将唯一ID映射回地址，实现双向查找



模式4：关系映射

```
mapping(address => address[]) followers;  
mapping(address => mapping(address => bool))  
isFollowing;
```

实现一对多关系（如用户关注多个其他用户）和快速检查关系存在



模式5：计数器

```
mapping(address => uint) public balances;  
  
uint public totalSupply;
```

存储每个地址的余额，记录总供应量或总计数



设计要点

- 根据业务需求选择合适的模式
- 组合模式解决更复杂的问题
- 考虑Gas成本和数据访问频率
- 合理使用索引优化查询效率

Mapping和Struct最佳实践

设计原则

- ✔ 使用Struct组织复杂数据，提高代码可读性
- ✔ 使用Mapping实现O(1)快速查找
- ✔ 配合Array实现遍历所有数据项
- ✔ 添加exists字段明确标记记录是否存在
- ✔ 使用计数器追踪集合中元素数量

命名规范

📖 **Mapping**使用复数或"to": balances、allowances、addressTold

📦 **Struct**使用单数名词: User、Proposal、Token

💡 清晰的命名能提高代码可读性，减少误解

Gas优化

- ✔ 限制Array大小，避免Gas消耗过多
- ✔ 分批查询，降低单次交易Gas消耗
- ✔ 考虑数据访问频率，选择合适存储位置
- ✔ 外部函数优先使用calldata，内部函数使用memory

exists字段使用

⚠ 问题: Mapping中无法区分"值是默认值"和"从未设置"

✔ 解决方案: 在Struct中添加bool exists字段

示例:

```
struct User {  
    string name;  
    uint balance;  
    bool exists; // 明确标记是否存在  
}
```

知识点总结

Mapping特性

-  键值对存储结构
类似HashMap或Dictionary
-  O(1)查找速度
哈希函数计算存储位置
-  所有键都"存在"
未设置键返回默认值
-  不存储键
无法获取键的列表
-  只能用于storage
不能在函数内创建

Struct特性

-  自定义复合数据类型
组织相关数据字段
-  三种创建方式
逐个赋值、构造器、键值对
-  可在不同存储位置
storage、memory、calldata
-  可作为参数和返回值
灵活的数据传递
-  可包含Mapping
仅能在storage中

组合模式

-  Mapping + Struct
快速查找复杂数据
-  Mapping + Array
可遍历的Mapping
-  exists字段
明确标记记录是否存在
-  完整组合模式
Mapping+Struct+Array
-  最佳实践
命名规范、Gas优化

掌握组合模式 = 掌握数据设计精髓

今日作业

任务1：用户管理系统

- ✓ 定义User结构体，包含：**name**、**email**、**balance**、**registeredAt**、**exists**
- ✓ 使用mapping存储用户信息：**mapping(address => User) users**
- ✓ 使用array存储地址列表：**address[] userAddresses**
- ✓ 实现功能：
 - register(name, email) - 注册
 - updateProfile(name, email) - 更新
 - deposit() payable - 存款
 - getUserInfo(address) - 查询
 - getAllUsers() - 获取所有用户
 - getUsersByRange(start, end) - 分批查询

⚠ 限制最多**1000**个用户

任务2：投票系统

- ✓ 定义Proposal结构体，包含**voters**的**mapping**
- ✓ 支持创建提案
- ✓ 支持投票（每人只能投一次）
- ✓ 查询提案信息
- ✓ 获取获胜提案
- 💡 思考：如何确保公平投票？
- 💡 思考：如何防止重复投票？

任务3：思考题

1. 为什么**mapping**不能遍历？
提示：考虑storage结构
2. 如何实现可遍历的**mapping**？
提示：结合array使用
3. **struct**中包含**mapping**有什么限制？
提示：storage位置相关
4. **mapping** + **struct**组合的优势是什么？
提示：数据组织与查找效率

完成作业并提交至github上

实战案例：用户管理系统

需求分析

- ✓ 存储用户信息（名字、余额、注册时间）
- ✓ 能够快速查找特定用户
- ✓ 能够遍历所有注册用户
- ✓ 能够检查用户是否已注册

数据结构设计

```
// 用户信息结构体
struct UserInfo {
    string name;
    uint balance;
    uint registeredAt;
    bool exists;
}

// 主存储Mapping
mapping (address => UserInfo) public users;

// 键列表Array
public userAddresses;
    address[]

// 计数器
public userCount;
uint
```

功能设计

- 🔧 register(string memory name)
- 🔍 getUserInfo(address userAddr)
- 👤 getAllUsers()
- ✓ isRegistered(address userAddr)
- 💰 deposit() payable

实现要点

🧩 使用Struct组织用户信息

📖 使用Mapping快速查找用户

使用Array遍历所有用户

下节课预告+结束

📅 下节课预告

第4.1课：控制流语句



条件语句

if-else、三元运算符



循环语句

for、while、do-while、break和continue



错误处理

require、assert、revert、try-catch

敬请期待!

📌 核心要点回顾



Mapping

快速查找，不能遍历



Struct

组织数据，提高可读性



组合模式

Mapping+Struct+Array



exists字段

明确标记存在

掌握组合模式 = 掌握数据设计精髓